

1. What is an array and for what general types of task might an array be useful?

A collection of values in one variable. They are useful for tasks that would require a list of related items.

2. Are TypeScript arrays indexed with integers or strings?

Integers starting at 0.

3. What is an array literal and how would you create one?

An array literal is the written form of an array. It is created by putting values inside square brackets separated with commas. For example:

```
let ages: number[] = [11, 12, 13, 14];
```

4. Is it possible for an array to be empty? If so, how would you represent an empty array and would that empty array be considered truthy or falsy? What characteristic of TypeScript arrays makes an empty array useful?

Yes it is possible. You would represent it with empty brackets: []. It is truthy. Empty arrays are useful because TypeScript arrays can grow and have elements added to them later.

5. What is an array type and how are array types defined?

An array type describes the type of values that an array can have. Array types can be defined using `type[]` or `Array<type>`.

6. Is there a built-in mechanism for converting strings into arrays? If so, explain the relevant syntax.

Yes. The `split()` method. It can divide a string into an array based on a "separator". Syntax:

```
let text: string = "Josh,Mike,Nate";  
let names: string[] = text.split(","); <- The comma is the separator
```

7. Is there a built-in mechanism for converting arrays into strings? If so, explain the relevant syntax.

Yes. The `join()` method. It is also based on a separator. Syntax:

```
let names: string[] = ["Josh", "Mike", "Nate"];  
let text: string = names.join(", ");
```

8. What is the syntax for creating a new array?

You can use square brackets or the array constructor. Syntax example:

```
let teams: string[] = [];  
let teams2: string[] = new Array<string>();
```

9. How do you determine the size of an array? Can the size of an array be changed after it has been introduced? If so, what is the syntax for doing so?

You determine the size by using the length property. It can be changed after it is introduced. You do this by assigning a new value to its length property using `array.length = newSize`. Making the length smaller removes elements from the end, while making it larger creates empty positions in the array. Example:

```
teams.length = 10;  
teams.length = 9; <- Makes it smaller
```

10. Is it possible to remove an arbitrary element from within an existing array? If so, what is the syntax for doing so?

Yes, with the `splice()` method. You specify the starting index you want removed then the number of elements and it removes it. Syntax:

```
let numbers: number[] = [10, 20, 30, 40];  
numbers.splice(2, 1); <- Removes "30" at index 2.
```

11. How can you add a new element to the beginning of an array? What about the end? How can you remove and return the element at the beginning of an array? What about the end?

You use `unshift()` to add an element to the beginning and `push()` to add an element to the end. `shift()` removes and returns the first element and `pop()` removes and returns the last element.

12. How can you add or remove a collection of elements from an arbitrary location within an array (specified as a starting index and a number of elements)? What if you simply want to set a range of elements all to the same value? What if you wanted to extract the values of a range of elements without actually changing the original array?

You use the `splice()` method to add or remove elements at a specific position, the `fill()` method can set a range of elements to the same value, and the `slice()` method can copy a range of elements without changing the original array.

13. Some methods of the Array object are considered mutating while others are considered non-mutating. Explain this distinction and why it is important. How does this help clarify the distinction between the Array object's `slice()` and `splice()` methods?

A mutating method changes the original array and a non-mutating method does not change the original array. This difference is important because changing the original data may affect other parts of a program. The object `splice()` is mutating, while `slice()` is non-mutating.

14. What does the spread operator do in the context of an array? How do you write one? How might one be useful?

The spread operator is written as ... and expands the individual elements of an array. It can be useful for copying or combining arrays. Example:

```
let first: number[] = [1, 2];  
let second: number[] = [3, 4];  
let combined: number[] = [...first, ...second];
```

15. The Array object has a concat() method. What does this method accomplish, and why has it become less frequently used in light of recent additions to the language?

The concat() method combines arrays and returns a new one. It's used less frequently because the spread operator provides a shorter and more readable way of combining arrays.

16. Explain the difference in behavior between the for...in loop and the for...of loop when applied to an array.

A for...in loop goes through the indexes of an array while a for...of loop goes through the actual values stored in the array.

17. What is meant by the concept of an iterator?

An iterator is an object that gives values from a collection one at a time.

18. What is meant by the concept of an iterable? What items are considered iterables? Why is this concept useful?

An iterable is an object whose values can be accessed one at a time by using an iterator. Iterables are useful because they can be used with features such as for...of loops and the spread operator.

Items considered iterables:

- **Arrays**
- **Strings**
- **Maps**
- **Sets**
- **Typed arrays**
- **Generator objects**

19. Explain the behavior of the Array object's keys(), values(), and entries() methods?

The keys() method returns an iterator with the indexes of an array, the values() method returns an iterator with the values, and the entries() method returns index-value pairs.

20. What is an array-like object? How does this concept relate to arrays and iterables? Which method of the Array object would you use to convert an array-like object into an

actual array?

An array-like object is an object that has numbered elements and a length property. It is not necessarily an actual array. Some array-like objects can also be iterable. `Array.from()` can be used to convert an array-like object or iterable into an actual array.

21. What is destructuring in the context of arrays? What is the syntax for employing it? When might it be useful?

Destructuring lets you take values from an array and put them into separate variables. It is useful when you want to get specific values from an array without having to use the indexes every time. Syntax:

```
let names: string[] = ["Josh", "Mike", "Nate"];  
let [first, second] = names;
```

22. From the JavaScript perspective, is an array a separate data type? If not, what is it? How can you determine if a value is an array? Does this change at all from the TypeScript perspective? If so, how?

In JS an array is not its own separate data type. It is considered a type of object. You can check if something is an array by using `Array.isArray()`. In TypeScript, arrays still work the same way at runtime, but TypeScript also lets you define what type of values the array is allowed to hold.

23. What is a tuple type and how are tuple types defined?

A tuple is like an array but each position has a type of value. Tuple types are defined by listing the types in order inside square brackets.

24. Describe some ways that the clean code philosophy could be meaningfully applied to the material you've discovered in this unit.

The clean code philosophy can be used with arrays by giving arrays and variables clear names, using the correct array method for the task at hand, and keeping the code simple instead of making it more complicated than it needs to be. TS types can also help make the code easier to understand by clearly showing what kind of values an array should contain.